IKB42603 – CLOUD COMPUTING SECURITY ESSENTIALS

Lab 1: Identity and Access Management (IAM) & Kubernetes RBAC

Course Information

Course Name: IKB42603 – Cloud Computing Security Essentials

Lab: Lab 1 – Identity and Access Management (IAM) & Kubernetes RBAC

Instructor: Madam Adani

Student Name: Sicid Sukhra Abdirahman

Student ID: 52215124928

Date: 5 August

Objective

The objective of this lab is to understand the fundamental concepts of Identity and Access Management (IAM) and Role-Based Access Control (RBAC). The lab demonstrates how to create IAM users, groups, policies, and access keys using LocalStack, and how to configure Kubernetes namespaces, roles, and role bindings to enforce the principle of least privilege.

Task 1: Map the Cloud Identity Landscape

This task introduces the core Identity and Access Management (IAM) concepts used throughout the laboratory.

Concept	AWS Term	Purpose

All-powerful owner	Root User	Has full control over the AWS account and resources.
Human/Application identity	IAM User	Represents an individual user or application that requires access.
Permission bundle	IAM Policy	Defines allowed or denied actions.
Collection of users	IAM Group	Applies the same permissions to multiple users.
Temporary identity	IAM Role	Provides temporary permissions without permanent credentials.

Task 2: Create a Least-Privilege Admin

An administrative group was created and assigned the AdministratorAccess policy. A user named **CloudAdmin_Arkhus** was created and added to the **Admins** group.

Commands Used

aws iam create-group --group-name Admins

aws iam attach-group-policy --group-name Admins --policy-arn arn:aws:iam::aws:policy/AdministratorAccess

aws iam create-user --user-name CloudAdmin_Arkhus

aws iam add-user-to-group --group-name Admins --user-name CloudAdmin_Arkhus

aws iam get-group --group-name Admins

```
$ aws $EP iam add-user-to-group --group-name Admins --user-name CloudAdmin_Arkh
us

user@LAPTOP-SNO2HB52 MINGW64 ~
$ aws $EP iam get-group --group-name Admins
{
    "Users": [
        {
            "Path": "/",
            "UserName": "CloudAdmin_Arkhus",
            "UserId": "atmgg8jm1ej2mehfrrir",
            "Arn": "arn:aws:iam::000000000000:user/CloudAdmin_Arkhus",
            "CreateDate": "2026-08-05T12:11:22.546000+00:00"
        }
    ],
    "Group": {
        "Path": "/",
        "GroupName": "Admins",
        "GroupId": "pxeshwdp5bh7Omrvzahy",
        "Arn": "arn:aws:iam::000000000000:group/Admins",
        "CreateDate": "2026-08-05T12:05:28.228000+00:00"
```

Result

The CloudAdmin_Arkhus user was successfully added to the Admins group.

Task 3: Enforce Least Privilege with a Scoped Policy

A read-only IAM user was created and assigned the AmazonS3ReadOnlyAccess managed policy.

Commands Used

:: aws iam create-user --user-name Analyst_Arkhus.

aws iam attach-user-policy --user-name Analyst_Arkhus --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

aws iam list-attached-user-policies --user-name Analyst_Arkhus

```
user@LAPTOP-SNO2HB52 MINGW64 ~
$ aws $EP iam list-attached-user-policies --user-name Analyst_Arkhus
{
    "AttachedPolicies": [
        {
            "PolicyName": "AmazonS3ReadOnlyAccess",
            "PolicyArn": "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
        }
    ]
}
```

Result

The Analyst_Arkhus user was successfully assigned the AmazonS3ReadOnlyAccess policy.

Task 4: Credential Hygiene and Access Keys

An access key was created for the analyst user and then updated to inactive to demonstrate access key management.

Commands Used

aws iam create-access-key --user-name Analyst_Arkhus

aws iam list-access-keys --user-name Analyst_Arkhus

aws iam update-access-key --user-name Analyst_Arkhus --access-key-id <AccessKeyID> --status Inactive

```
user@LAPTOP-SNO2HB52 MINGW64 ~
$ aws $EP iam list-access-keys --user-name Analyst_Arkhus
{
    "AccessKeyMetadata": [
        {
            "UserName": "Analyst_Arkhus",
            "AccessKeyId": "LKIAQAAAAAAAM6KIEN3O",
            "Status": "Inactive",
            "CreateDate": "2026-08-05T12:38:10+00:00"
        }
    ]
}


user@LAPTOP-SNO2HB52 MINGW64 ~
$
```

Result

The access key was successfully created, listed, and updated to an inactive state.

Task 5: Separate Environments with Namespaces

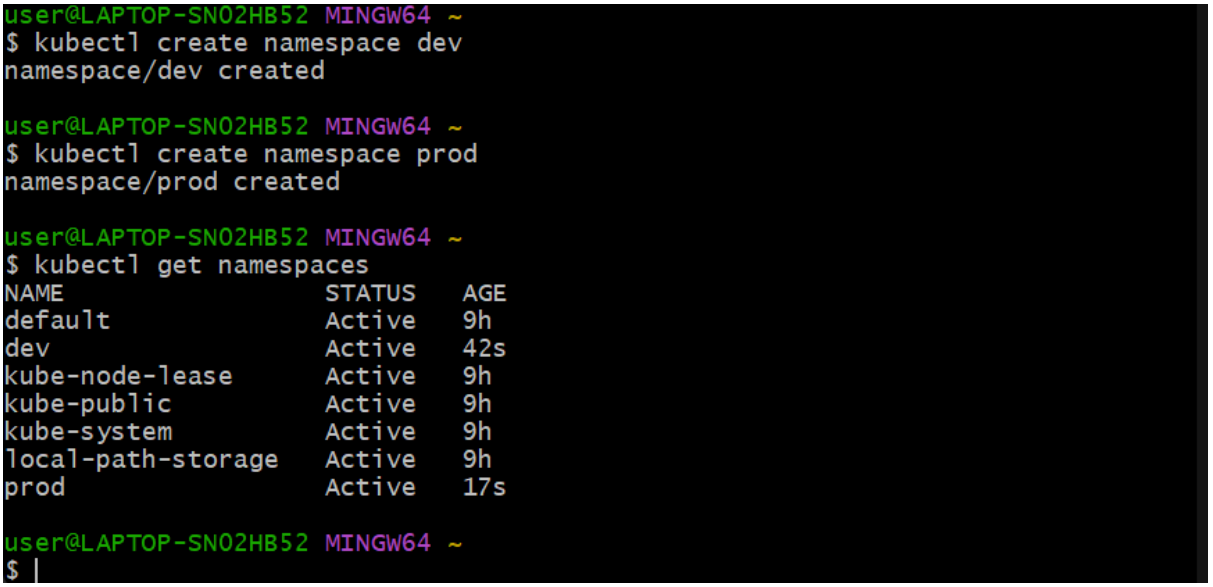
Two Kubernetes namespaces were created to separate development and production environments.

Commands Used

:: kubectl create namespace dev

kubectl create namespace prod

kubectl get namespaces



Result

The dev and prod namespaces were successfully created and verified.

Task 6: Define a Role and Bind It

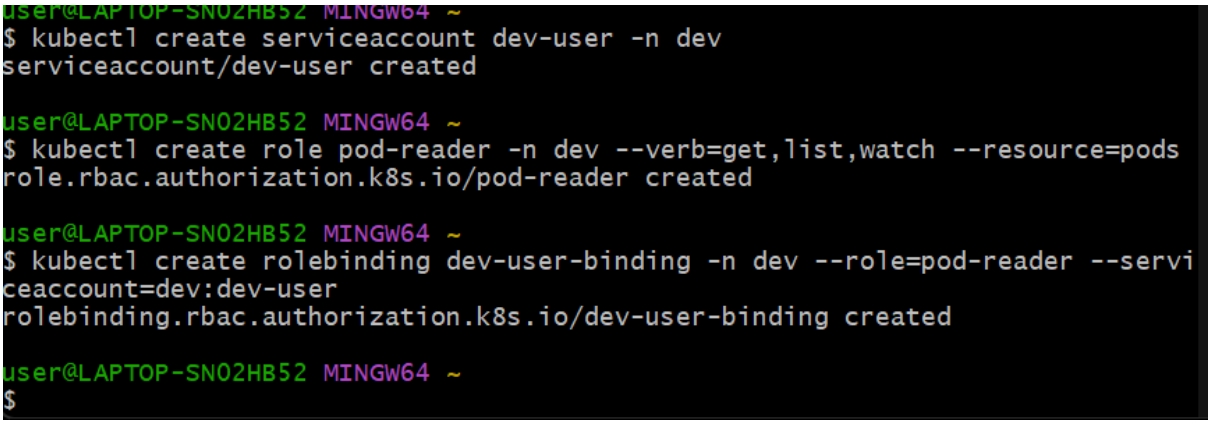
A service account, role, and role binding were created within the dev namespace to implement least privilege.

Commands Used

:: kubectl create serviceaccount dev-user -n dev.

kubectl create role pod-reader -n dev --verb=get,list,watch --resource=pods

kubectl create rolebinding dev-user-binding -n dev --role=pod-reader --serviceaccount=dev:dev-user



Result

The service account, role, and role binding were successfully created.

Task 7: Test That Access Control Works

The permissions assigned through RBAC were verified using Kubernetes authorization checks.

Commands Used

kubectl auth can-i list pods -n dev --as=$SA

kubectl auth can-i delete pods -n dev --as=$SA

kubectl auth can-i list pods -n prod --as=$SA

Verification Command

kubectl get rolebinding dev-user-binding -n dev -o yaml

```
user@LAPTOP-SN02HB52 MINGW64 ~
$ kubectl get rolebinding dev-user-binding -n dev -o yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  creationTimestamp: "2026-08-05T12:49:59Z"
  name: dev-user-binding
  namespace: dev
  resourceVersion: "11496"
  uid: eca9a9ac-4768-468e-a1d7-f33df1c7d729
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: pod-reader
subjects:
- kind: ServiceAccount
  name: dev-user
  namespace: dev

user@LAPTOP-SN02HB52 MINGW64 ~
```

Result

The RBAC configuration worked as expected. The service account was allowed to list pods in the dev namespace but was denied permission to delete pods or access resources in the prod namespace.

Conclusion

This lab demonstrated the implementation of Identity and Access Management (IAM) and Kubernetes Role-Based Access Control (RBAC) in a local cloud environment. IAM users, groups, policies, and access keys were successfully configured using LocalStack. Kubernetes namespaces, roles, service accounts, and role bindings were created and tested to enforce the principle of least privilege. The successful verification confirmed that access permissions were correctly applied in both AWS IAM and Kubernetes RBAC.

Reflection Questions

Q1. Why is attaching policies to groups better than attaching them directly to users?

Answer:

Attaching policies to groups makes permission management easier. When a policy is attached to a group, all users in that group automatically receive the same permissions. This saves time, reduces mistakes, and makes access easier to manage.

Q2. What is the difference between an IAM User and an IAM Role?

Answer:

An IAM User is a permanent identity with its own login credentials or access keys. An IAM Role does not have permanent credentials. It provides temporary permissions that can be assumed by users, applications, or AWS services when needed.

Q3. Explain least privilege using the Analyst account, and how it reduces blast radius if compromised.

Answer:

The Analyst account was given only the **AmazonS3ReadOnlyAccess** policy. It can only read data from Amazon S3 and cannot modify or delete resources. If the account is compromised, the attacker has only limited permissions, reducing the amount of damage that can be caused.

Q4. In Kubernetes, what is the difference between a Role and a RoleBinding?

Answer:

A Role defines what actions are allowed on Kubernetes resources within a namespace. A RoleBinding connects that Role to a user or service account, allowing that identity to use the permissions defined in the Role.

Q5. Why did the developer service account fail to access prod, and which security principle does that demonstrate?

Answer:

The **dev-user** service account could not access the **prod** namespace because its RoleBinding was created only in the **dev** namespace. It had no permissions in **prod**. This demonstrates the **principle of least privilege**, where users receive only the permissions they need to perform their tasks.